SUM, SUMX, AVERAGE, AVERAGEX, ROW-WISE CALCULATIONS & RUNNING TOTALS IN POWER BI

DAX (Data Analysis Expressions) is the core calculation language used in Power BI, Power Pivot, and SSAS Tabular models. Even though DAX looks simple at first—using functions like SUM, AVERAGE, DIVIDE, etc.—its true power is visible only when we understand how it handles **row context vs filter context**, and how iterative functions such as **SUMX, AVERAGEX** evaluate expressions **row by row** over a table.

The example of calculating **profitability percentage** (Profit ÷ Revenue) illustrates a fundamental difference between **aggregated calculations** and **row-wise (iterative) calculations**. These concepts also apply to running totals, cumulative sums, window functions, and advanced DAX patterns.

This note provides a comprehensive explanation of:

1. The difference between **implicit and explicit measures**
2. The behavior of **SUM** vs **SUMX**
3. How percentage measures behave in totals
4. Why SUMX solves the row-wise calculation problem
5. The difference between **AVERAGE** and **AVERAGEX**
6. How to calculate profitability measures correctly in different scenarios
7. How to compute running totals using DAX
8. Important errors, pitfalls, and best practices

YATICORP
EMPOWERING EDUCATION

---

1. Implicit vs Explicit Measures (Quick Recap)

Power BI automatically creates **implicit measures** when you drag a numeric field into a visual (e.g., SUM of Revenue).
But when you manually use DAX to create a new calculation—like:

```
Profitability = SUM(ProfitAmount) / SUM(RevenueAmount)
```

—that is called an **explicit measure**.

Explicit measures always obey filter context and are more flexible and powerful

2. Understanding the First Profitability Measure

You initially created:

```
Profitability (%) =
    DIVIDE( SUM(SalesData[ProfitAmount]), SUM(SalesData[RevenueAmount]) )
```

Key properties:

- SUM creates a **single aggregated number** for ProfitAmount
- SUM creates another **aggregated number** for RevenueAmount
- The measure divides the two totals

Therefore:

For each row in a table visual, the calculation is:

TOTAL Profit (in current filter) ÷ TOTAL Revenue (in current filter)

If the visual is filtered at the row level (by a single OrderID), the measure behaves correctly.

But for the **Grand Total** row, the visual filter contains **all rows**.
Thus, the DAX engine does:

Grand Total = SUM(all profits) ÷ SUM(all revenues)

This is **not the same as**:

Sum of row-wise percentages

This problem appears in many business KPIs such as:

- Profit margin
- Win percentage
- Conversion rate
- Discounts
- Ratios of any type

YATICORP
EMPOWERING EDUCATION

Totals of ratios **never** equal ratio of totals unless all denominators are equal.

---

3. Why SUM Cannot Perform Row-Wise Calculations

The SUM function works only on **columns**, not expressions.

Example:

```
SUM(SalesData[ProfitAmount] / SalesData[RevenueAmount])
```

This is not allowed because DAX does not sum expressions row-by-row unless you explicitly instruct it to.

This leads to the need for **SUMX**.

---

4. SUMX – The Row-Wise Iterator

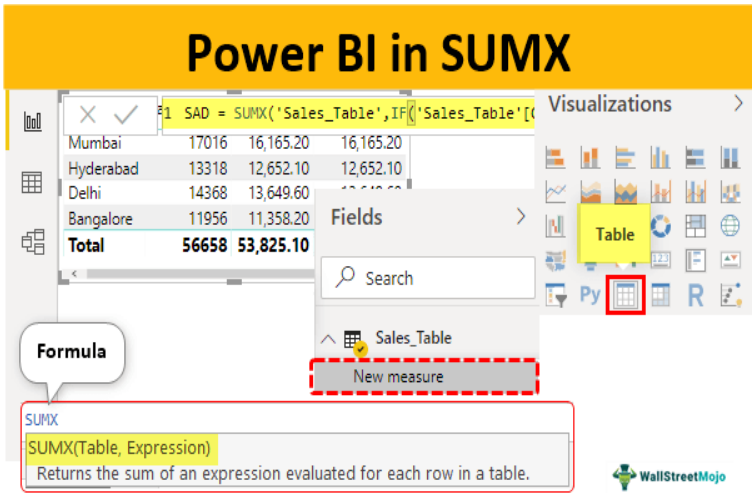
SUMX syntax:

```
SUMX ( Table, Expression )
```

YATCORP
EMPOWERING EDUCATION

SUMX performs the following:

1. Iterates through **each row** of the specified table
2. Evaluates the expression for that row
3. Adds all evaluated results



The measure you created:

```
Sum of Profitability RowWise =
   SUMX(
      SalesData,
      SalesData[ProfitAmount] / SalesData[RevenueAmount]
   )
```

What happens internally:

- For each OrderID row:
    - Take ProfitAmount of that row
    - Take RevenueAmount of that row
    - Compute row_profitability = profit/revenue
- After evaluating all rows, add them:

Grand Total = Σ (profitability for each row)

Important effect:

- The *individual rows* appear identical between the SUM and SUMX measures
- The *grand total* differs dramatically

This difference is the **most important power of SUMX**.

YATICORP
EMPOWERING EDUCATION

---

5. Why SUMX Is Needed for Ratios and Percentages

Whenever you compute:

- Ratio
- Percentage
- Weighted average
- Row-wise computed value

SUM will never give correct results because it aggregates first and calculates later.

SUMX evaluates each row first, then aggregates later.

General rule:

Use SUM for aggregated column totals.
Use SUMX when you must perform a calculation for each row before aggregation.

6. Example: Why the Grand Total from SUM is Wrong

Your original profitability:

Total Profit = 7314
Total Revenue = 13890
Profitability = 7314 / 13890 ≈ 5.59%

But individual orders may have profitability:

4%, 10%, 12%, 20%, etc.

Adding these gives a much larger total than 5.59%.

Hence, two measures produce:

- **Profitability using SUM** → 5.59%
- **Profitability using SUMX** → sum of all row-wise percentages (e.g., 800%, depending on data)

Both results are mathematically correct—but serve different purposes.

YATICORP
EMPOWERING EDUCATION

7. AVERAGE vs AVERAGEX

You also calculated the average profitability.

Attempt using AVERAGE:

Average Profitability = AVERAGE(Profitability)

This fails because:

- AVERAGE only works on **columns**
- Profitability is a **measure**, not a column

Therefore AVERAGE cannot see it.

Correct approach: AVERAGEX

Like SUMX, AVERAGEX performs row-wise evaluation:

```
AVG Profitability (%) =
   AVERAGEX(
      SalesData,
      SalesData[ProfitAmount] / SalesData[RevenueAmount]
   )
```

This correctly gives a true average of profitability across orders.

Alternatively (two-step method)

Create profitability measure:

```
Profitability =
   DIVIDE( SUM(SalesData[ProfitAmount]),
         SUM(SalesData[RevenueAmount]) )
```

Then create average:

```
Avg Profitability = AVERAGEX( SalesData, [Profitability] )
```

YATICORP
EMPOWERING EDUCATION

Either approach works.

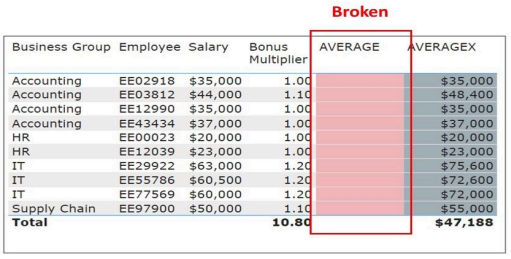
---

8. When to Use AVERAGE vs AVERAGEX

Use AVERAGE when:

- You want average of a column (without row-wise expressions)
- Example: average revenue per transaction





Business Group	Employee	Salary	Bonus Multiplier	AVERAGE	AVERAGEX
Accounting	EE02918	$35,000	1.00		$35,000
Accounting	EE03812	$44,000	1.10		$48,400
Accounting	EE12990	$35,000	1.00		$35,000
Accounting	EE43434	$37,000	1.00		$37,000
HR	EE00023	$20,000	1.00		$20,000
HR	EE12039	$23,000	1.00		$23,000
IT	EE29922	$63,000	1.20		$75,600
IT	EE55786	$60,500	1.20		$72,600
IT	EE77569	$60,000	1.20		$72,000
Supply Chain	EE97900	$50,000	1.10		$55,000
Total			**10.80**		**$47,188**

Use AVERAGEX when:

- You need to average calculated expressions
- You need access to row-level values
- You want to average ratios, percentages, KPIs, or calculations not stored in a column

9. Understanding X Functions (Iterators)

All iterative (row-wise) functions in DAX end with **X**:

- SUMX
- AVERAGEX
- MINX
- MAXX
- COUNTX
- RANKX
- PRODUCTX
- CONCATENATEX

YATICORP
EMPOWERING EDUCATION

These functions:

- Loop through a table
- Apply row context
- Evaluate the expression per row
- Combine the results

This is different from standard functions like SUM, AVERAGE, MIN, etc., which work only on columns.

---

10. Row Context vs Filter Context

Row Context

- Operates on a single row at a time
- Exists in calculated columns and iterator functions (SUMX, AVERAGEX)

Filter Context

- Applies filters based on the visual (e.g., row = OrderID 1001)
- Every measure is evaluated under filter context

Implication:

SUMX introduces a *row-by-row* calculation that SUM cannot perform.

YATICORP
EMPOWERING EDUCATION

11. Creating Running Totals (Cumulative Totals)

Running total or cumulative total is widely used in:

- Daily revenue accumulation
- Monthly cumulative sales
- Year-to-date totals
- Running count of records

Given your example with SalesDate and Revenue:

Basic running total measure

```
Running Revenue =
CALCULATE(
    SUM(SalesData[Revenue]),
    FILTER(
        ALL(SalesData[SalesDate]),
        SalesData[SalesDate] <= MAX(SalesData[SalesDate])
    )
)
```

Using TOTALYTD

If you want year-to-date totals:

```
Revenue YTD =
TOTALYTD(
    SUM(SalesData[Revenue]),
    SalesData[SalesDate]
)
```

YATICORP
EMPOWERING EDUCATION

Other variants:

- TOTALMTD (month-to-date)
- TOTALQTD (quarter-to-date)

Your version: Using TOTALYTD-like syntax

```
Cumulative Revenue =
TOTALYTD(
    SUM(SalesData[Revenue]),
    SalesData[SalesDate]
)
```

You used a similar approach.

Why Running Totals Must Use CALCULATE or Time Intelligence

Because a running total requires:

- Removing the current filter on dates
- Re-applying a custom filter such as "all dates up to current date"

CALCULATE modifies the filter context.

FILTER allows specifying the custom logic.

---

12. Important Issues with Date Hierarchy

Power BI automatically creates:

- Year
- Quarter
- Month
- Day hierarchy

YATICORP
EMPOWERING EDUCATION

If you don't remove it, running totals will behave unpredictably, because the hierarchy groups the data by year, then month, etc.

You correctly:

1. Turned off the hierarchy by selecting the raw date field
2. Changed the formatting to dd-mm-yyyy or desired format

This ensures that the running total is computed by individual dates, not grouped hierarchically.

---

13. The Mathematical Reason Behind Incorrect Totals for Percentages

Let each row i have:

profitability_i = profit_i / revenue_i

The correct average profitability is:

(Σ profit_i / revenue_i) / count

But the aggregated measure gives:

SUM(profit) / SUM(revenue)

These two are equal **only** when all revenues are identical.

Thus:

- SUM-based profitability gives a **weighted average** based on revenue.
- SUMX-based profitability or AVERAGEX gives a **simple average**.

Both may be needed depending on business requirements.

YATICORP
EMPOWERING EDUCATION

---

14. Practical Use Cases of SUMX and AVERAGEX

Use SUMX for:

- Profit margin per order (Profit / Revenue)
- Tax per row (Revenue * TaxRate)
- Discounts (Quantity * Discount%)
- Weighted KPIs
- Interest calculations (Principal * Rate)
- Cost-per-unit calculations

Use AVERAGEX for:

- Average discount rate
- Average margin across orders
- Average customer satisfaction score
- Average yield per transaction

---

15. Best Practices for DAX in Such Scenarios

✔ Always name measures clearly

Use prefixes like:

- Total Revenue
- Total Profit
- Margin (%)
- Margin RowWise
- Avg Margin (%)

YATICORP
EMPOWERING EDUCATION

✔ Avoid creating unnecessary intermediate measures unless required

In your example, you avoided creating separate profitability measure when using AVERAGEX.
This keeps your model clean.

✔ Use DIVIDE instead of /

DIVIDE handles divide-by-zero errors gracefully.

DIVIDE(numerator, denominator, alternateResult)

✔ Use SUMX for any calculation that uses two or more columns together

✔ Test totals separately

Totals row may behave differently from individual rows

16. Summary of Key Lessons (Executive Summary)

1. **SUM** aggregates column values.
2. **SUMX** iterates row-by-row, evaluating expressions per row.
3. **SUM-based profitability** gives a weighted average (total profit ÷ total revenue).
4. **SUMX and AVERAGEX profitability** give row-level calculations such as sum of percentages or average of percentages.
5. Always use **iterator functions** for ratio-based KPIs.
6. **AVERAGE cannot average a measure**, but **AVERAGEX can** because it works on expressions.
7. Running totals require manipulating filter context using **CALCULATE + FILTER** or **TOTALYTD**.
8. Disable automatic date hierarchies when working with true running totals.
9. DAX works differently in the "Total" row because the filter context is broader.
10. Choose SUM or SUMX based on business requirement—both are correct for different purposes.

YATICORP
EMPOWERING EDUCATION